INTERNSHIP PROJECT · TÜRKSAT AVIATION AND COMMUNICATION

Smart School Information System

Week 3 — Rule Engine (Role-Based Access Control)

PREPARED BY	Bera Ergin — s29315
INSTITUTION	PJAIT — 7th Semester
THIS WEEK	The rule engine that decides who can see what

Where We're Coming From, Where We're Going

WEEK 2 — DONE

- ✓ FastAPI + JWT auth working
- ✓ All 4 roles log in correctly
- ✓ Token carries role/related_id
- ✓ But no endpoint returns data yet

The token proves identity but doesn't restrict anything yet.

WEEK 3 — GOAL

- RuleEngine class — generating filters per role
- Teacher → own class, parent → own child
- Bound parameters (against SQL injection)
- Isolated unit + integration tests

Not wired to any API endpoint yet — the rule engine is tested on its own.

RuleEngine Design

An independent filter rule for each role — four lines of logic, but the foundation of all the security

Principal	principal	No filter — all records	`1=1`
Teacher	teacher	Only their own class	`class_name = :class_name`
Parent	parent	Only their own child	`id = :student_id`
Student	student	Only themselves	`id = :student_id`

Note: parent and student filters look identical because a parent account's related_id already stores the child's student id.

Why Bound Parameters

Parameter binding instead of string concatenation — making SQL injection impossible by design

DANGEROUS — string concat

```
f"SELECT * FROM students
WHERE class_name = '{related_class}'"
```

If related_class arrives as "9-A' OR '1'='1", the filter is completely bypassed.

SAFE — bound parameter

```
"SELECT * FROM students
WHERE class_name = :class_name"
params = {"class_name": related_class}
```

related_class is no longer part of the SQL — it's plain data. The database driver never mixes the value into the query text.

RuleEngine never builds SQL with f-strings or %-formatting under any condition — every value goes through SQLAlchemy's text()/bindparam() mechanism.

Test Strategy

Two layers: the rule itself, and its behavior against a real database

Unit tests

- Does each role produce the correct WHERE clause?
- Does a missing related_class/related_id raise ValueError?
- Is an unknown role (e.g. 'hacker') rejected?
- Does the generated clause contain only :params, no raw values?

Integration tests

- Does teacher1 see only 9-A in the real SQLite data?
- Does parent1 see only their own child?
- Does principal see all 12 students?
- Is an SQL injection attempt (') treated as plain data?

Sample Test Output

```
pytest backend/tests/test_rule_engine.py -v
```

```
test_principal_sees_all                          PASSED
test_teacher_filters_by_class                    PASSED
test_parent_filters_by_child                     PASSED
test_student_filters_by_self                     PASSED
test_teacher_without_class_raises                PASSED
test_unknown_role_raises                         PASSED
test_integration_teacher_sees_only_9A            PASSED
test_integration_parent_sees_only_child          PASSED
test_sql_injection_attempt_is_literal            PASSED
```

```
9 passed in 0.14s
```

Dependencies Added This Week

requirements-dev.txt — testing only, not needed in production

`pytest`	Test runner
`pytest-cov`	Coverage reporting

Carried over from Week 2: fastapi, uvicorn, python-jose, passlib (unchanged)

Week 3 — Scope and Acceptance Criteria

Delivered this week

- ✓ backend/app/services/rule_engine.py — RuleEngine class
- ✓ Bound-parameter SQL filter for all 4 roles
- ✓ backend/tests/test_rule_engine.py — unit tests
- ✓ Integration tests (against real SQLite data)
- ✓ A test covering an SQL injection attempt
- ✓ requirements-dev.txt (pytest, pytest-cov)

Acceptance criteria

1. pytest backend/tests/ — all tests PASSED
2. Teacher's filter contains only their own related_class
3. Parent/Student filter contains only their own related_id
4. No raw string interpolation anywhere in the generated SQL
5. RuleEngine is not yet wired to any FastAPI route

Next Step — Week 4

Natural Language → SQL (LLM Integration with Claude)

- POST /query endpoint — the first endpoint that returns real data
- Translating a natural-language question into SQL with Claude
- Passing the generated SQL through RuleEngine
- A safety check that allows only SELECT statements

Thank you